

Eötvös Loránd University

Faculty of Informatics

Department of Artificial Intelligence

Addressing the limited context window problem of transformer-based language models with graph databases

Advisor:

Dr. László Csaba Gulyás,
Associate Professor,
Department of Artificial Intelligence

Author:

Ákos András Prucs
Computer Science
Department of Artificial Intelligence

Budapest, 2024

Table of Contents

Table of Contents 2
Chapter 1: Introduction 1
1.1 Research Goals Overview 4
1.1.1 Concepts 5
1.2 Personal Objectives and Learning Outcomes 5
Chapter 2: Related Work 7
2.1 Enhancing Conversational AI with Memory Integration 7
2.2 Graph-Based Data Integration in Language Models 7
2.3 Language models used in the project 8
2.4 Classical Relation Extraction Pipelines 9
2.5 Relation Extraction By End-to-end Language Generation (REBEL) 10
2.6 Dolly Dataset 12
2.7 WebNLG Dataset 13
2.8 Rouge Score 14
Chapter 3: Methodology 16
3.1 Single LLM with Multiple Conversational Agents 16
3.1.1 Instruction Fine-Tuned Models 17
3.1.2 Chat Templates 19
3.2 Utilizing REBEL for Triplet Extraction 20
3.3 Custom Triplet Extractor Model 21
3.4 Conversational model fine-tuning 23
Chapter 4: Results 26
4.1 Conversational Agent System 26
4.2 Align the model to custom chat templates 28
4.3 Quality of Custom Relation Extractor 28
4.4 Conversational Agent on Triplet Comprehension 30
4.5 Model Responses on Cherry Picked Task 34
Chapter 5: Conclusions 37
References 38
List of Figures 43
List of Tables 46
Appendix A: Environment Configuration 47

A.1 Flash Attention [42] 47
A.2 Quantization [43] 49
A.3 PEFT and LoRA..... 51

Chapter 1: Introduction

The evolution of language models (LMs) is an essential phenomenon that has developed natural language processing (NLP) and artificial intelligence (AI). In the beginning, classical LMs used statistical methods, like n-gram models [1], which calculated the conditional probability of the next word in a sequence of words by determining the occurrence of such sequences in a given textual corpus. The simplest variant is a unigram, which considers each item in isolation. While bigram looks at pairs of consecutive items. With the use of n-grams, one can estimate the probability of each item in a sequence, conditionally based on its preceding n-1 items.

N.garm models succeeded with common short-word patterns, but the model was ineffective at long-range dependencies and the subtleties of human language usage.

However, the emergence of neural network-based techniques marked a significant shift in the paradigm. Neural language models, based on recurrent neural networks (RNNs) [2] and later, Long Short-Term Memory (LSTM) [3] networks, marked a significant improvement by being able to remember information for longer periods and by better handling the sequential nature of language.

Recurrent Neural Networks (RNNs) are a type of neural networks specifically intended to process sequential input, such as time series and text. Recurrent neural networks differ from feedforward neural networks in that they have internal loops that enable the retention of information. This is what allows them to integrate context from prior sections of the sequence when interpreting or predicting later ones. While Recurrent Neural Networks (RNNs) have the potential to retain long-range context according to theory, in reality, the problem of vanishing or bursting gradients arises as the length of the sequence increases. Long Short-Term Memory (LSTM) networks, among other types, have been suggested as solutions for addressing these issues.

LSTMs are a specific kind of RNNs that have been developed with the purpose of retaining information over long periods of time. LSTMs are distinguished by their

capacity to control the transmission of information by employing gate architectures. These gates have the ability to determine which data in a sequence should be retained or discarded, therefore resolving the issue of the vanishing gradient problem that is often encountered in conventional RNNs. LSTMs possess the ability to excel at tasks that require the consideration of long-term relationships, such as language translation, speech recognition, and time-series prediction.

The invention of the Transformer architecture [4] was another outstanding achievement that led to another paradigm shift. It abandoned recurrence and replaced it with a model that employs attention mechanisms to learn contextual relationships between words in a sentence without considering their positional distance.

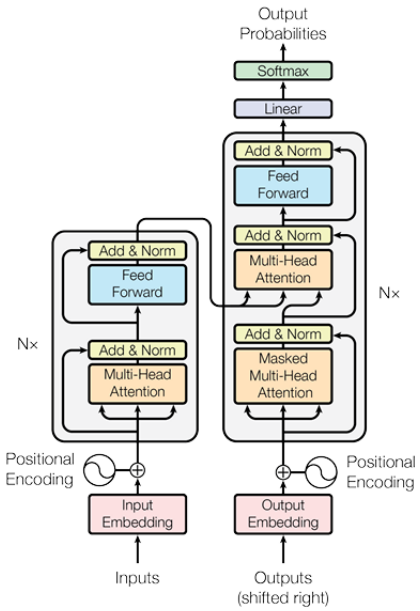
Attention mechanisms have revolutionised the way neural networks perceive and process sequences. The approach draws inspiration from human visual attention, where attention is focused dynamically over different parts of a scene to gather more information. In neural networks, attention mechanisms determine what parts of input data are relevant. This is very important for complex tasks, such as machine translation, in which the relevance of input elements may highly differ from one word to the next.

The Transformer architecture is a radical break from all the models proposed so far for processing sequences based on RNNs or LSTMs. The architecture [4] relies exclusively on mechanisms of self-attention to balance the importance of various words in a sentence, regardless of their distance in position. Unlike RNNs, Transformers can process all the words in a sentence simultaneously, making them very parallelisable and thereby much faster to train.

With this development being the groundwork for models such as GPT (Generative Pre-trained Transformer) [5] and BERT (Bidirectional Encoder Representations from Transformers) [6], the capability of language understanding and language generation has been remarkable. These state-of-the-art LMs, pre-trained on large datasets and fine-tuned for different tasks, provide dramatic efficiency improvements for various applications. Nowadays, neural language models are the main focus of AI research, trying to develop more ways to improve what machines can do when exposed to human language.

Figure 1.1 depicts the architecture of a Transformer model and the data flow from input embeddings to output probabilities with multiple layers of attention and normalisations. The embeddings convert categorical data, such as words, into a vector of real numbers representing the items in a computational space where distances between vectors are indicative of semantic similarity. Word embeddings in natural language processing allow for a dense representation that embeds similar words with similar encoding. They reduce data dimensionality when compared to one-hot encoded vectors and are able to capture syntactic and semantic properties of language meaningfully. Some methods for word embeddings include Word2Vec [7], GloVe [8], and FastText [9], among many others, which are trained on vast text corpora.

The main points presented in the figure are multi-head attention [10], which allows the model to attend to several input parts simultaneously, and positional encoding [11], which brings inputs into sequential order. Furthermore, this model's design tackles the typical limits of a fixed context window by permitting every fragment to potentially constrain the others across the whole sequence, thus improving the model's ability to handle more extended context when processing inputs and generating outputs.



1.1: The transformer model architecture (from the paper *[4]*)

Although neural language models are very powerful, especially when deployed in chatbot applications, they have some significant limitations that are mainly connected to their context window. The context window is the number of tokens or words the model can consider while generating its output. For instance, GPT-2 [12], GPT-3 [13] or any transformer-based language model have restricted context windows that limit their ability to stay contextually relevant and coherent during prolonged interactions. This constraint almost always leads to the loss of context in the long dialogues where the model cannot refer to data beyond the maximum size window. Therefore, dialogues may be disengaged or repeated, negatively affecting users and the model's capacity in practical scenarios.

Researchers have experimented with different techniques to tackle these challenges. One strategy, for instance, is based on external memory systems, which can store and output information acquired from previous interactions, thus expanding the context window [9] [10]. Another technique is to increase the adaptive attention span in neural networks [14] by appropriately selecting the critical parts of conversation history.

These changes have certainly contributed to an important domain of AI and natural language processing research, which undertakes to remove one of the fundamental models' limitations. Extending the frame of reference makes human-machine communication more natural and opens up the possibility of utilising language models in a much wider range of tasks, ranging from very complex to demanding.

1.1 Research Goals Overview

This thesis proposes a similar solution to those experimenting with external databases. This time, the main focus is external graph databases and their direct compatibility with language model text generation when data is injected into prompt as triplets. In this scenario, a triplet is a head, tail, and relationship combination representing two nodes and their connection in a graph. The resulting models are tested in particular text comprehension datasets, where the context of each question/task is converted into a list of triplets using text-to-graph models.

1.1.1 Concepts

External graph databases are databases that specialize in storing and managing graph structures. Graph databases are different from relational databases because they store data in nodes, edges, and properties instead of storing them in tables. This makes them very appropriate for applications involving complex relationships and dynamic schema, including social networks, recommendation systems, and the case of this thesis: enhancing language models with structured data.

A **triplet** [15] [16] refers to the set of three components: head, tail, and a relation. The head and tail denote the nodes in a graph, and the relation tells how they are connected. This construct is the very essence of representing semantic relationships in graph databases and stands very crucial for such data integration into language models.

Text-to-graph models [17] are specialized models or algorithms that map text data onto graph structures. By analyzing text and extracting entities and relationships, such models can develop triplets representing the information as a network of nodes and edges. This conversion is important if the graph databases are to be used for natural language processing tasks where the relational information between various text entities is relevant.

Language model text generation entails the use of machine learning models, more specifically language models, in the process of text generation. Language models are trained on huge text datasets and are made to understand what follows or doesn't follow in a sequence of words. Further, this ability is exploited to generate coherent text contextually relevant to a given input or prompt.

1.2 Personal Objectives and Learning Outcomes

This whole project has deeply involved personal development targets, which are definitely of great importance. The first aim is to get more familiar with language models and text generation techniques. This exploration, therefore, involves reading how these technologies work in terms of different algorithms and configurations that are essential

to achieving enhanced capabilities of generating natural texts. In this way, I will be able to manage such challenges and opportunities better as I better understand these models.

Then, moving on to another objective, which is to get to know Huggingface tools [18] and the enthusiastic community that helps in using and developing them. Huggingface has established itself as a key player in the machine learning field, mainly in natural language processing, by providing a comprehensive set of pre-trained models and libraries for the production of advanced language applications. Participating in discussions, contributing to ongoing projects, and learning from others in the huggingface community is more than using the tools; it also means being part of the discussion. Such an interaction will most likely sharpen my technical abilities and extend my view of the cutting-edge trends and techniques used in the development of AI.

Chapter 2: Related Work

2.1 Enhancing Conversational AI with Memory Integration

The MemGPT project [19] is a remarkably innovative endeavour that takes it one step further by overcoming the drawbacks of fixed context windows in neural language models. MemGPT addresses the context consistency issue over long conversations by integrating a memory module that significantly improves dialogue capabilities. This project's architecture utilises external memory synchronised with the core GPT model so that it can store conversation context and vital user data over several sessions.

MemGPT has been created with the purpose of remembering and utilising past conversations for future interactions, allowing for a more consistent and relevant exchange of information. This ability becomes particularly crucial when the conversations last several sessions and need the details of the earlier chats, such as customer service. Implementing MemGPT is a complex system of registration and restoration of conversation chunks to enable the model to rectify past interactions via conversational analysis. This, therefore, guarantees that every response is created in real time and supported by a thorough comprehension of the conversation's history.

2.2 Graph-Based Data Integration in Language Models

The fact that the MemGPT project was very successful in dealing with context windows by implementing a novel memory integration approach is one of the strongest factors that influenced my interest in improving the performance of the language models I develop. Observing this idea, my thesis plans to be similar to this, but it would be approached from a different perspective by using a graph-based infrastructure. The approach relies on storing and injecting structured data in the form of triplets into the conversation as graph databases, thus enabling the language model to cope with more intricate and elaborate dialogues with higher efficiency.

The use of graph databases brings a rare opportunity for organising data in a way that more closely represents the human cognitive processes than the traditional linear databases. Organising data as nodes and edges empowers the language model to accomplish read and write operations, which are similar to human memory retrieval and storage. This method increases the flexibility and relevance of the conversational AI's responses and enables it to retain the context throughout long conversations.

The main goal of my project is to convert the text into a graph-like format, which in turn is used to train the model to identify and generate additional text by this structured data representation. This requires the development of appropriate techniques for extracting triples from text – an important step that guarantees the quality and usability of the data going into the system. The challenges from a technical perspective will be associated with the model implementation process and ensuring it interacts smoothly with graph databases while not losing its ability to make sense of the context. By analysing this method, its feasibility, and its added value compared to existing models, my work may serve as a base for further research in this area.

2.3 Language models used in the project

I utilised different language models in order to analyse how the central hypothesis in my project is put to work. Through the process of inserting data sourced from external platforms, they would not have to rely too much on the lexical knowledge (parameter-based) developed internally [20]. In addition, I used the smaller parameter sizes of the popular models, such as Google's Gemma 2B [21], Meta's Llama 7B [22], and Mistral 7B [23]. These models were selected for their strong architecture while having a smaller parameter count compared to their bigger counterparts.

The reason for adopting this methodology is based on the assumption that the models can perform the tasks more accurately and efficiently if they have access to external data sources. Through the use of graph databases filled with the data represented as triples, the models are assigned the primary task of processing and utilising the provided external information to produce contextually relevant text. Through this procedure, the point is to

transfer the load from the parameters to the model's capacity to link and employ the data of a predetermined format as a dynamic resource. This transition poses a test in terms of the adaptability and flexibility of these models and is an experiment in reducing the computational burden usually associated with large-scale language models.

This novel implementation lays the foundation for the majority of my project design experimentation and gives us a chance to find out the ways to make language processing models more efficient and effective. The aim of this project is to work with smaller parameter models and external data to be able to engage in a dialogue on designing language models that both perform well and feature practical usability in the real world.

2.4 Classical Relation Extraction Pipelines

The traditional relation extraction pipelines [24] [25] [26] follow a multi-step process to identify semantic relationships between entities in text. The pipeline includes Named Entity Recognition (NER) techniques and Relation Classification (RC) methods. NER detects and classifies entities in the raw text and labels them as specific types. Meanwhile, RC is applied following NER to determine how entities are related typically based on predefined categories. While this pipeline effectively extracts structured information, such techniques tend to be fragile due to their dependency on the initial entity recognition phase. Also, they suffer from error propagation, where mistakes in early stages can affect the outcome of later stages. It is also worth mentioning that classical approaches are typically limited to a predefined set of entity and relation types, which makes it necessary to retrain to adapt to new, unseen categories.

Figure 3.2 illustrates an information extraction pipeline. Its components are closely related to the ones mentioned above. Coreference resolution gathers linguistic expressions (mentions) in a text that refer to the same entity. This is a common step after NER. Named Entity Linking (NEL) is about recognising names of persons, organisations or geographic terms that have an entry in an external knowledge base. A popular example of NEL is the sentence „Paris is the capital of France", where Paris and France can be linked to their Wikipedia pages.

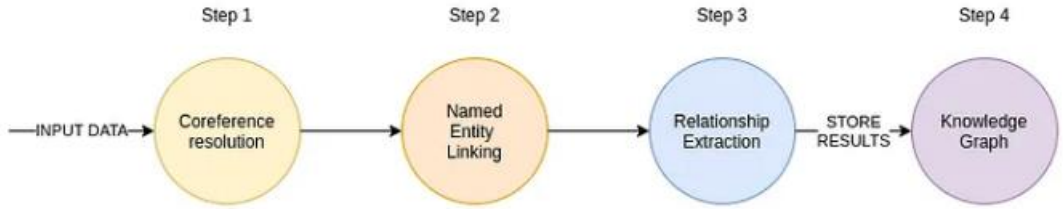


Figure 2.1 Illustration of a Natural Language Processing (NLP) pipeline. [27]

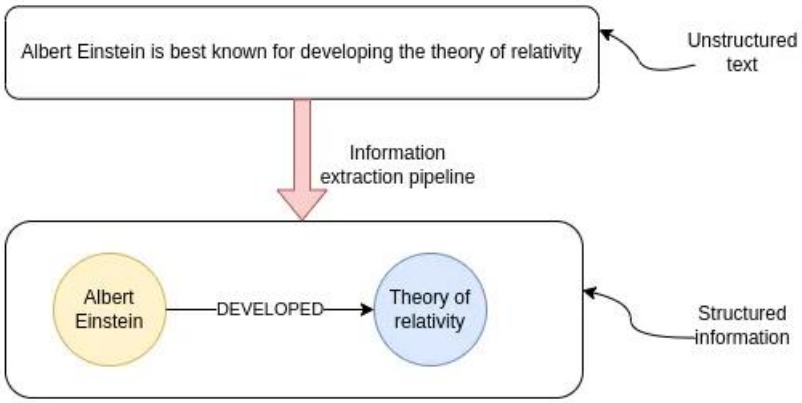


Figure 2.2 An example inference of a relation extraction pipeline. [27]

2.5 Relation Extraction By End-to-end Language Generation (REBEL)

In 2021, the Babelscape team proposed a novel approach for relation extraction tasks [28]. Instead of using multiple-step pipelines and relying on the risks of propagation errors, they employed an end-to-end text generation model, making it possible to perform the relation extraction directly, focusing on the raw text. They created REBEL by finetuning the BART model, which is a language model pre-trained by denoising sequence-to-sequence tasks.

Their text generation model treats extracted relations as text sequences and train the model to format the output according to a unique prompt template (Their algorithm for

prompt generation can be seen in Figure 3.4. An example data point that is transformed into the predefined prompt structure can be seen in Figure 3.5). This allows them to read out the predicted relations from the formatted prompt using regex methods. Their solution can also handle more than 200 relation types, making it flexible for never-seen text.

Result:
$lin_triplets$ with all triplets as a sequence of text.
Input:
E = Entities;
R = Relations;
$sort()$ Sorts by placement in input text;
Start:
$E = sort(E)$;
$lin_triplets$ = "";
for $e \in E$ **do**
  $R(e)$ = relations with e as subject;
  $R(e) = sort(R(e))$;
  $lin_triplets$ += <triplet> + e;
  for $r \in R(e)$ **do**
    $o = E(e, r)$ object of relation r;
    $lin_triplets$ += <subj> + o + <obj> + r;
  end
end

Figure 2.3 REBEL's algorithm to transform a set of triplets into text format. This transformation is necessary to create a valid output set for fine-tuning the BART model. Notice the special tokens <triplet>, <subj> and <obj> that form a unique prompt template. [28]

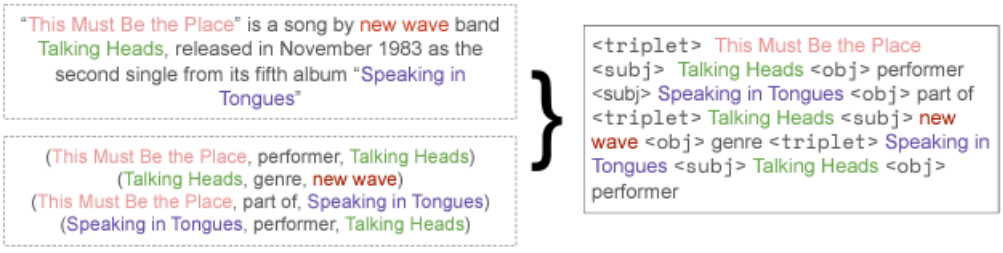


Figure 2.4 The sentence in the top-left corner is the model's original x input. Below that is the set of triplets corresponding to the input. On the right is the transformed text version of the relation set used to train the model. [28]

2.6 Dolly Dataset

The dataset is described on the website of the authors [27], and further information can be found in the InstructGPT paper [28] and its hugging face page [29]. The dataset (originally called databricks-dolly-15k) consists of over 15,000 records generated by thousands of Databricks employees. Each record is a prompt / response pair, and they can be split into seven different categories, namely:

- **Creative Writing:** A question or instruction that requires a creative, open-ended written response. The instruction is reasonable for a person with general world knowledge and does not require searching. The prompt gives very explicit instructions to follow.
- **Closed QA:** A question or instruction that demands a factually correct response based on a passage of material from Wikipedia. The question can be complicated and can include human-level reasoning ability but does not require particular knowledge.
- **Open QA:** A question that can be answered using general world knowledge or, at most, a single search. This job asks for opinions and facts regarding the world at large.
- **Summarization:** A summary of a paragraph from Wikipedia. It does not contain questions requiring more than 3-5 minutes to answer.
- **Information Extraction:** These questions involve reading a paragraph from Wikipedia and extracting information from the passage. Everything required to produce an answer (e.g. a list, keywords, etc) is included in the passages.
- **Classification:** These prompts contain lists or examples of entities to be classified, e.g. movie reviews, products, etc. In this task, the text or list of entities under consideration is contained in the prompt.

- **Brainstorming:** Think up many examples in response to a question asking to brainstorm ideas.

instruction	context	response
Given a reference text about Run Towards the Danger, tell me how many essays are part of the collection.	Run Towards the Danger is a 2022 Canadian essay collection by Sarah Polley, a former child star, director, and screenwriter. The six essays in the collection examine aspects of Polley's career on stage, screen, and on film detailing her roles in a Stratford Festival production of Alice Through the Looking Glass, as well as her breakout roles in The Adventures of Baron Munchausen and the TV series Road to Avonlea. The book also revealed for the first time that Polley had been a victim of Jian Ghomeshi who sexually and physically assaulted her when she was 16 and he was 28.	Six essays are part of the Run Towards the Danger essay collection.
What is the last festival celebrated in Tihar and how long does the festival last?	Lakshmi Puja is celebrated as a part of Tihar, a second national festival of Nepal after Dashain. In Nepal, it is celebrated for five days, which include Kag (crow) Tihar; Kukur (dog) Tihar; Gai (cow) Tihar in the morning and Lakshmi Puja at night; Maha puja (self puja); Goru (Ox and Bull) Tihar and Gobardhan puja; and finally, Bhai Tika (Bhai dhooj)—respectively the first, second, third, fourth and fifth days.	The festival lasts for 5 days and Bhai Tika is the last festival.
If each client was served by the same number of professionals, how many professionals (round to one decimal place) would each client be served by?	RGP, formerly known as Resources Global Professionals, is the operating arm of Resources Connection, Inc. (NASDAQ:RGP). The company provides consulting services in the areas of finance & accounting; information management; governance, risk & compliance (GRC); human capital; legal & regulatory; corporate advisory & restructuring; strategic communications; and supply chain management. As of fiscal year ending May 28, 2016, the company employed 3,283 professionals in 68 offices in 20 countries around the world. The company reported serving over 1,700 clients. Company headquarters are located in Irvine, CA.	3283 professionals and 1700 clients, so 3283/1700 = 1.9 professionals serving each client.

Figure 2.5 Datapoints from databricks-dolly-15k

2.7 WebNLG Dataset

The WebNLG [30] [31] collection consists of triplet sets that describe facts (entities and their relations) and the matching facts expressed in natural language text. There are up to seven triplet sets in the corpus, and each set has one or more reference texts. The dataset was first utilised for the WebNLG challenge, which involves mapping the triplet sets to text and performing tasks like surface realisation, lexicalisation, aggregation, production of referencing expressions, and sentence segmentation. The corpus is also utilised for the extraction of triplets in reverse.

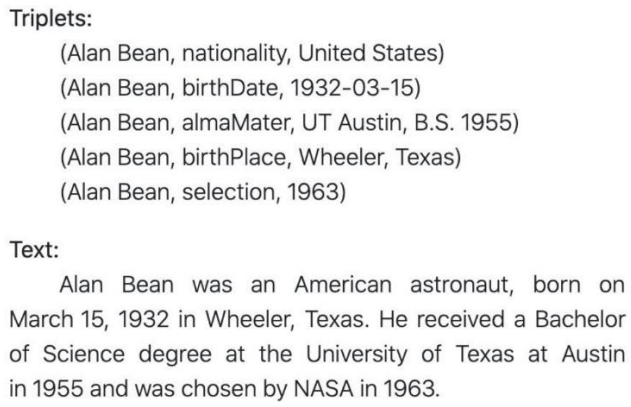


Figure 2.6 An instance from the WebNLG dataset containing the written sentence and the corresponding triplet representation of its factual information. (figure from the website [31])

2.8 Rouge Score

ROUGE [32] [33] is an abbreviation for Recall-Oriented Understudy for Gisting Evaluation and is a set of metrics used to evaluate the quality of summaries generated by automated systems against human-made summaries. The ROUGE score is mainly used to measure the n-gram overlap between the machine-generated text and the reference text, which is given by humans. This evaluation method is considered one of the best in the area of natural language processing, especially for tasks of text summarisation and machine translation in which the structure and fidelity of the generated text are critical. ROUGE metrics are advantageous because they allow a quantitative measurement of the extent to which the main aspects of the human reference summaries are retained by the automated summaries.

ROUGE-1 and ROUGE-2 scores are used to evaluate the number of times a single word and two consecutive words appear in the generated sentence. A high ROUGE-1 score measures the number of words from the reference summary that are found in the generated summary, which suggests good content coverage. A high ROUGE-2 score, on the other side, means that the summary has the right content and maintains the degree of similarity of phrases and structure to the reference, which is a sign that the details and the context have not been lost. ROUGE-L, which uses the LCS (longest common

subsequence) metric, is a measure of the longest sequence of words that appear in the same order in both the generated and reference summaries. This score is most useful for assessing the sentence level fluency and order of information in the summary, allowing us to measure the structural coherence relative to the original text.

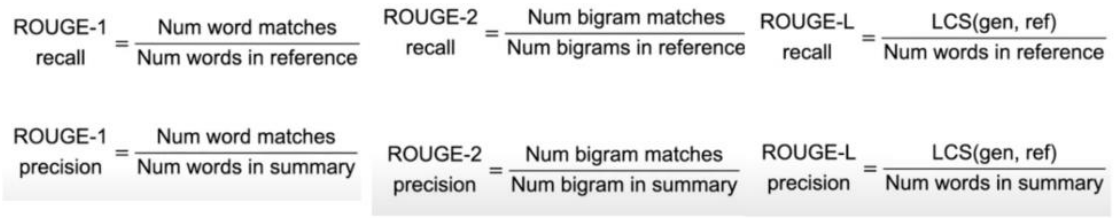


Figure 2.7 Formulas for calculating different variations of ROUGE. Note that LCS stands for longest common sequence. (Image taken from this video: [34])

Chapter 3: Methodology

In this thesis, I propose three approaches to aligning text generation models to triplet comprehension to test out how they would work with a hypothetical graph database:

1. A multi-agent system having 3 agents with unique roles, each processing the input text in a different way: **a,** A text-to-triplet role saving extra information into a graph database. **b,** A text-to-keywords role making queries to the database based on user input. **c,** A conversational agent receiving the user input and the queried external information is input. This implementation is inspired by what MemGPT proposed, having multiple chatbots with different roles.
2. Modified version of the first implementation by utilising specialised text-to-graph models. Both text-to-graph and text-to-keywords roles are replaced with a single model, namely REBEL, being able to identify more than 200 relation types.
3. This implementation uses a fine-tuned text-to-graph text generation model that replaces REBEL. The fine-tuning is done on the WebNLG dataset composed of sentence and triplet pairs. To make the model learning more efficient, I further modified the dataset to contain diverse sentences and triplets per data point.

Text to graph models are not only nececcary to support conversational models by providing them useful text-to-graph information, but they are also needed to generate evaluation datasets for triplet comprehenshion task.

3.1 Single LLM with Multiple Conversational Agents

The first implementation proposes solution closely resembling the MemGPT [19] model. This involves developing three agents, each designed with a specific function in mind. An agent operates as a distinct inference method with a unique prompt and context tailored to its designated role. The primary responsibilities assigned to these agents include:

- Extracting valuable triplet data and keywords from user inputs is crucial for enriching the conversational context and enhancing the model's responsiveness.

- Responding accurately to users' questions, facilitating a seamless and interactive dialogue experience.

The composition of these agents is also extended with an external graph database. Using the keywords extracted by one of the agents, triplet data from a graph database is queried and then appended to the assistant agent's prompt.

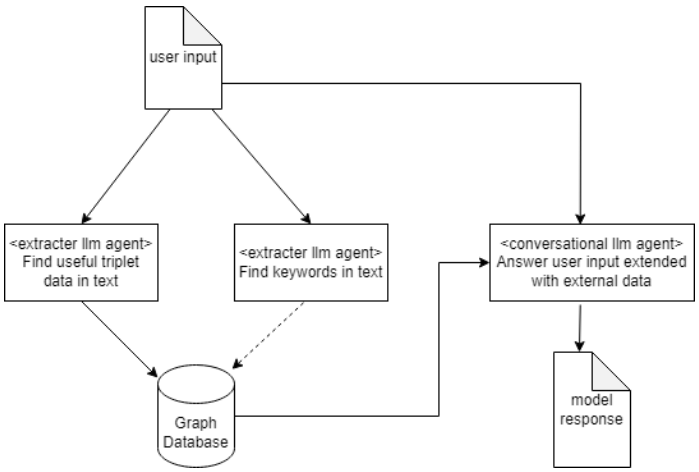


Figure 3.1 The diagram illustrates the architecture of the multi-agent system mentioned, which is designed to enhance conversational AI by integrating with an external graph database. It shows three specialised agents: the first agent extracts useful triplet data from user input, the second identifies keywords, and the third, a conversational AI agent, uses the extracted information to generate informed responses.

3.1.1 Instruction Fine-Tuned Models

Instruct models [35] [36] are designed to adhere closely to user instructions, enhancing their responses' precision and applicability. Unlike traditional language models that

generate replies based on predictive text generation, instruct models are trained to accurately parse and execute specific user commands or queries.

This training involves a deliberate and structured approach where the models are conditioned to recognise and prioritise the intent behind user inputs. The instruct models are typically fine-tuned from pre-trained language models using datasets specifically curated to include a variety of commands and actionable requests. This process imbues them with the capability to discern not just the content but also the context and urgency of user instructions.

Integrating instruction models into our conversational AI framework involves crafting detailed and directive-oriented prompt templates that guide the model's response generation. In Chapter 4.2, I will discuss instruction fine-tuning itself, as I fine-tune models as part of the project. In Chapter 5, I will show evaluation results on how instruction fine-tuned models solve the problems more accurately than simple pre-trained ones.

Table 3.1 Propmt templates for the agents. There are instructions describing their task and how to process the user input. Notice that I used one-shot prompts in these examples. Also, each prompt has a frame with special tokens; this is further discussed in Chat Templates.

agent	prompt templates
triplet extractor agent	### Instruction: *Analyse the user input. Extract identifiable information and express it as triplets suitable for database storage. Each triplet should capture a key entity, another related entity, and their relationship based on the context provided. Format your output as follows: <entity1>,<relationship>,<entity2>.* ### User message: …
keyword extractor agent	### Instruction: *Examine the user input to determine if further information from the database is necessary for an effective response. If so, identify and list keywords that will be used to query the database for relevant information. For instance, if the user asks, 'How to care for an orchid?', your response should be orchid \n care instructions.* ### User message: …

conversational agent	### Instruction: *You have received a message from the user. Additionally, relevant information has been retrieved from the database. Using this information, converse with the user to solve their problem or answer their query.* ### Data Triplets: … ### User message: …

3.1.2 Chat Templates

Chat templates are predefined structures used to guide the flow of conversation within language model agents [37]. They serve as a framework for the agent's responses, ensuring that the interactions remain consistent, contextually relevant, and aligned with the agent's intended functionality. Essentially, these templates dictate how responses are formulated based on input, incorporating any necessary variables or data retrieved during the interaction. The purpose of chat templates is to maintain a coherent dialogue and enhance the user experience by making interactions smoother and more predictable.

Developing unique templates for each model is often beneficial when using different pre-trained large language models (LLMs). This customisation is due to variations in the models' training data, design, and inherent capabilities, which can significantly influence how they interpret and generate language. By tailoring templates to each model's specific characteristics and strengths, developers can maximise the efficiency and effectiveness of their conversational agents, ensuring that each model performs optimally within the framework of its designated tasks.

Table 3.2 Recommended templates for using the specified models as conversational agents. Notice that each template incorporates specific formatting and special tokens to structure interactions appropriately. The <bos> and <eos> tokens represent the beginning and end of sequences, helping the models to delineate the start and end of user and model turns effectively. Special tokens such as [INST] and <<SYS>> are used to indicate instructional or system-specific prompts [38] [39].

model family	chat template

Google/Gemma	<bos><start_of_turn>user { user_msg_1 }<end_of_turn> <start_of_turn>model { model_answer_1 }<end_of_turn><eos>
Meta/Llama-2	<bos><s>[INST] <<SYS>> {{ system_prompt }} <</SYS>> [/INST]</s> { user_msg_1 }[/INST] { model_answer_1 } </s><eos>

3.2 Utilizing REBEL for Triplet Extraction

The key change compared to the first implementation is the replacement of the keyword extractor and relation extractor agents. Note that these agents worked under the same language model but were prompted uniquely according to their role in the system. Instead of using these agents, I use REBEL, a relation extraction model discussed in Relation Extraction By End-to-end Language Generation (REBEL). The reason behind this idea is that the conversational model depends on the quality of the injected triplets. Therefore, better relations may increase performance.

The expectation is to extract better-quality relations from the text, therefore increasing the conversational model's effectiveness.

Hypotetical communication with the database happens by inserting triplets and querying on keywords. With a relation extractrion model, triplets are provided, but getting the right keywords is not trivial. This implementation uses the head and tail components of each triplet as keywords.

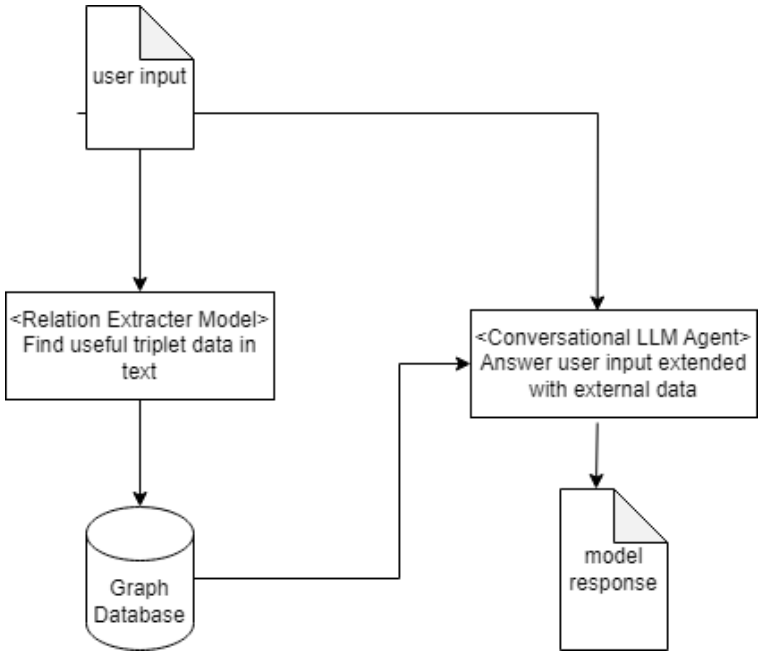


Figure 3.2 shows the Architecture of the second implementation. Notice that keyword extraction from text happens by querying each part of the triplets.

3.3 Custom Triplet Extractor Model

The third implementation doesn't include any change in the main conversational model. The focus is rather on the quality of the triplets extracted. Since REBEL is limited to over 200 relation categories, a custom-trained relation extractor is being utilised instead.

Training a relation extraction model includes taking a pre-trained language model and fine-tuning it on a triplet dataset. This time, training is carried out on the WebNLG dataset, which focuses on converting factual information into relations in the form of triplets.

Initially, a data point consists of a sentence (or multiple wording of it) and a set of triplets representing the factual information stored by the sentence. As my use case requires processing longer texts than a single sentence, it is necessary to make the dataset harder.

During preprocessing, I combined multiple data points by concatenating the sentences together. The distribution of the sentences' quantity for 1, 2, 3, 4 and 5 pieces is 40%, 20%, 20%, 10% and 10%.

However, combining the triplets is not so trivial, because for N triplets, there is N factorial possible ordering of them. By utilising random ordering, I would create an extra error score during model training. This would possibly slow down or prevent the model from learning. My solution is to order the collection of triplets in grammatical order. An example of the formatted input and output columns (text and relations) is shown in Figure 3.3.

Text:
AMC Matador is also known as American Motors Matador which is a mid sized car and equipped with an AMC V8 engine.
The car is assembled in Thames, New Zealand.
The United States Air Force who are the operating organisation for Al Asad airbase fought in the Korean war and at the invasion of Panama.
GMA New Media, which is located in Quezon City, makes web sites and online games.

Triplets:
AMC_Matador alternativeName "American Motors Matador"
AMC_Matador assembly Thames,_New_Zealand
AMC_Matador class Mid-size_car
AMC_Matador engine AMC_V8_engine
Al_Asad_Airbase operatingOrganisation United_States_Air_Force
GMA_New_Media location Quezon_City
GMA_New_Media product Online_Game
GMA_New_Media product World_Wide_Web
United_States_Air_Force battle Korean_War
United_States_Air_Force battle United_States_invasion_of_Panama

Figure 3.3 Sample datapoint of the preprocessed WebNLG dataset with combined sentences. Notice the grammatical order of the triplets.

The prompt template is also designed for the extractor model. The prompt here is divided into two sections. In the first part, the input text takes place with the special *### Text:* token indicating the place of the sentences. The second part of the prompt is noted with the *### Relations:* token, indicating the start of the triplets' section. Furthermore, each triplet is inside a frame of tokens: <entry>, <head>, <rel> and <tail>. I place the triplets

inside these frames so that it is easier to extract them with regex methods after the model aligns on these token patterns.

It is worth mentioning that a special data collator is also utilised during learning to guide the model training. The *completionsOnlyCollator* (implemented by [40]) is popular when it comes to instruction fine-tuning. With this collator I can disable calculating weights for token generation at a specified part of the prompt. In detail, the model does not learn by predicting tokens in the first half of the prompt, where the input text is defined. Error calculation is live for those parts of the prompt that come after the *### Relations:* special token where the desired output is inserted. An example of the input prompt is shown in Figure 3.4.

Text:
Peter Stoger is currently at FK Austria Wien and manages FC Koln, he also plays international football for Austria. The Greenlandic language is spoken in Denmark where Aarhus airport is located in Tirstrup, part of the Central Denmark region. Addiction (journal) has the abbreviation of "Addiction" and an LCCN number of 93645978.

Relations:
<entry><head>1._FC_Köln<rel>manager<tail>Peter_Stöger
<entry><head>Aarhus_Airport<rel>location<tail>Tirstrup
<entry><head>Addiction_(journal)<rel>LCCN_number<tail>93645978
<entry><head>Addiction_(journal)<rel>abbreviation<tail>"Addiction"
<entry><head>Denmark<rel>language<tail>Greenlandic_language
<entry><head>Peter_Stöger<rel>club<tail>Austria_national_football_team
<entry><head>Peter_Stöger<rel>club<tail>FK_Austria_Wien
<entry><head>Tirstrup<rel>country<tail>Denmark
<entry><head>Tirstrup<rel>isPartOf<tail>Central_Denmark_Region<eos>

Figure 3.4 This is a sample prompt from the fine-tuning of the relation extractor model. Notice the special tokens: ### Text:, ### Relations:, <entry>, <head>, <tail>, <rel>. During fine-tuning the model, weights are calculated for the generated token sequences only after the ### Relations: token.

3.4 Conversational model fine-tuning

Even though the application works this way, the conversational agent can be further optimised. The underlying LLM model were not trained to handle triplet data, nor do it know about the unique prompt template where the user instruction, the triplets, and the model's response are put in. Fine-tuning is necessary to align the model to my new

standard. I am using the Databricks/Dolly-15k dataset introduced in Chapter 2.6 for this purpose (and later on, I use this very same dataset for evaluating my models).

Dolly's data points are categorised by type of task. Some tasks have a context attached to a question. These are mainly text comprehension tasks: Closed QA, Information Extraction, and Summarization. I use them to create a new triplet comprehension task by transforming the contexts of each data point using one of the relation extractor models. (Example shown in Figure 3.5)

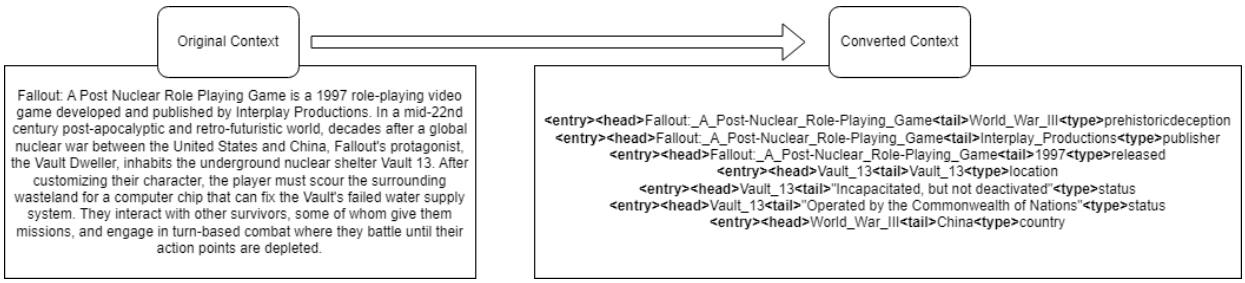


Figure 3.5 A sample data point from Dolly's context field was transferred into a set of triplets, forming a new dataset.

This newly generated dataset allows me to **a)** further fine-tune the conversational models and **b)** evaluate the performance of each model in a triplet comprehension task.

When training the models, I utilised two different collators to configure the training process further. With the *full collator*, the text generation training works normally; the model generates tokens and weights are calculated for each token in the sequence. This will, of course, generate further error scores during training due to the never-seen input text.

By using the *completions-only collator* (already discussed in Chapter 3.3, but this time I am fine-tuning the conversational model for triplet comprehension.), I can configure the training in such a way that weights will only be calculated for a specified part of the prompt. In this case, I turned off weights at the first half of the prompt, where the user

instruction and set of triplets are, and only calculated weights based on what the model would normally generate during inference. Note that a prompt here is referenced as the entire text sequence, including the model input and output as well.

Chapter 4: Results

4.1 Conversational Agent System

Certain challenges became evident when utilising conversational agents using pre-trained language models such as Gemma and Llama. While highly effective for a broad range of language understanding and generation tasks, these models are not specifically optimised for the extraction of triplet-like data from text. This is not a surprise, as these models were not trained on text-to-graph-like tasks. These challenges complicated controlling triplet and keyword extraction, having the models deviate from the specified output formats.

However, the fine-tuned conversational agent showed improvements in utilising triplets in the prompt. A figure that highlights the different responses from pre-trained and fine-tuned models is included below.



*Figure 4.1 pre-trained **google/gemma-2b-it** model's response to a instruction prompt with triplets inserted.*



Figure 4.2 ***Fine-tuned Google/gemma-2b-it*** *model's response to an instruction prompt. Notice that the response contains a word from the provided triplets (Amazon biome) in this aligned version. This indicates that the aligned model utilises the externally injected data.*

In the example shown, the input prompt for the models is a very simple question about where the Amazon rainforest is. Structured triplet data is added to this, linking the Amazon rainforest to the more general Amazon biome; this gives extra relational context that the models could use to construct a response.

The figures depict both the pre-trained Google model, gemma-2b-it, and its aligned counterpart. The two responses from each model show a clear difference between how each of these models reacts to the injected triplet data. The former pre-trained model, with no tuning regarding this type of structured data usage, makes a more generalised and descriptive response, "The Amazon rainforest is located in South America, in the western hemisphere…" and locates it in its place on the continent. While this response is correct, it does not leverage the relational data in the triplets directly.

In contrast, the fine-tuned variant of the same model, directly refers to the "Amazon biome" in the answer. It clearly uses the provided triplet information when locating the Amazon rainforest "in the Amazon biome, South America". This example shows that the fine-tuning of the model on triplet data tasks could strengthen its ability to integrate and express structured information in its responses.

4.2 Align the model to custom chat templates

Google's Gemma 2b-it model is easily aligned to custom prompts and unique triplet representations. Figure 4.3 shows the error scores during the fine-tuning phase. As the diagram indicates, around 30 learning steps were enough to align the model, as further training barely improved the error rate. In this fine-tuning phase, the Gemma-2b-it model was aligned with my custom triplet comprehension task made by Dolly. Performance scores are shown in Table 4.1.

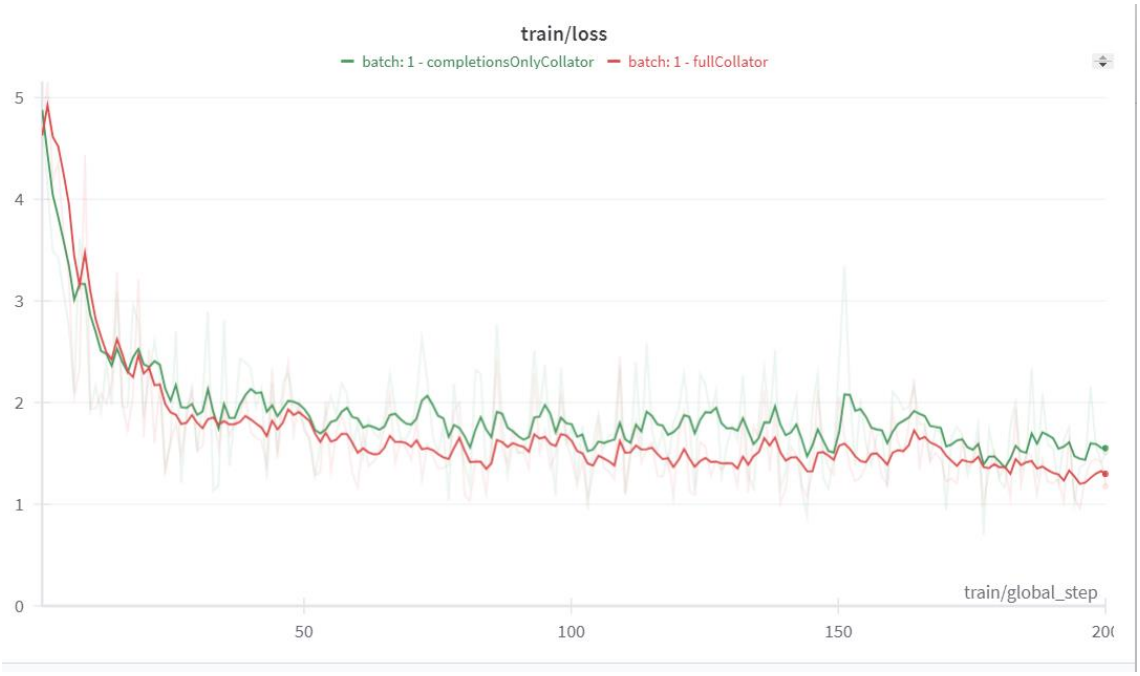


Figure 4.3 Error scores during fine-tuning on Dolly. The graph contains a comparison between two training runs using a complete collator (red) and a completions-only collator (green). Notice that the models aligned after 30 learning steps. (Plot created in WandB [41])

4.3 Quality of Custom Relation Extractor

Training the Google/Gemma 2b-it text generation model on triplet generation was surprisingly effective. Even with the harder custom dataset created from the original WebNLG's data, the model's predicted triplets were semantically accurate to the original

triplets. Differences between generation and true outputs are mostly due to the use of different words or signings in text representations. The model was fine-tuned for 5k training steps of the total 13200 data points using single batches. This means the model did not see duplicate datapoints during learning.

As shown in Figure 4.4, even after 5k training steps the extractor model was still learning on never seen datapoints.

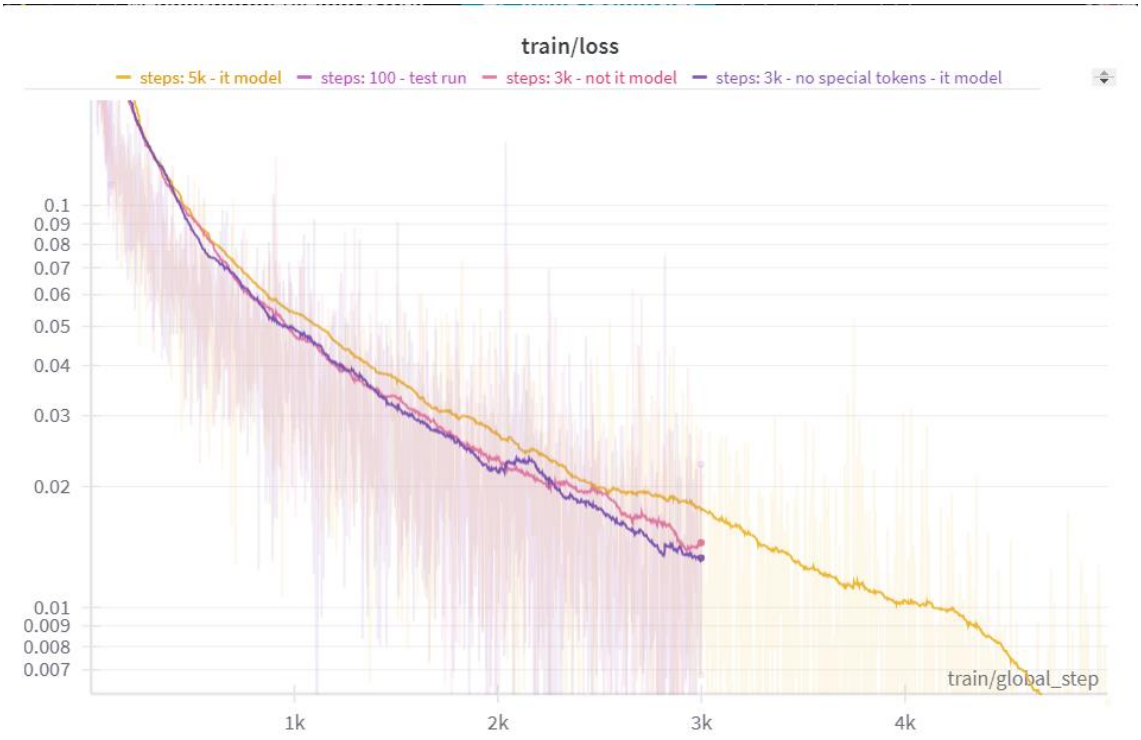


Figure 4.4 shows the training loss for the Google/Gemma 2b-it model over 5000 steps in a logarithmic scale to demonstrate how the model performance improves while training on a custom dataset created based on the WebNLG data. The loss values provided by the graph imply that the model is constantly learning and improving as the training continues. (Plot created in WandB [41])

This adoption of a transformer-based text generation model as relation extractor was a major shift in the project. The WebNLG dataset contains 372 unique relation types, which is close to twice as much as REBEL's categories. Furthermore, the original model is a pre-trained text generation model, just as REBEL, therefore defining categories for the

model is a flexible process as it is not overtuned on the dataset. This makes the triplet comprehension task easier by having contexts that contain triplets with more eloquent words.

Table 4.1 Comparison of the relation extracters: REBEL and the custom trained extractor on WebNLG. The table shows their extracted relations from the first paragraph of Napoleon I's Wikipedia page [42].

REBEL			**custom trained extractor on WebNLG**		
head	***relation***	***tail***	***head***	***relation***	***tail***
Napoleon Bonaparte	participant in	French Revolution	Napoleon Bonaparte	alternativeName	Napoleón Bonaparte
	conflict	Revolutionary Wars		birthName	Napoleone di Buonaparte
French Revolution	participant	Napoleon Bonaparte		deathDate	1821-05-05
Revolutionary Wars	participant			deathPlace	France
				emperor	French_Empire
					French_Republic
				militaryCommander	French_Army
					Revolutionary_Wars
				modernizing	France
				regnalName	Napoleon I
				revolutionaryYears	1804-1815
				revolution	French

4.4 Conversational Agent on Triplet Comprehension

Models are evaluated using the triplet comprehension tasks generated from Dolly, from which 200-200 data points are selected into a test set. Since there are contexts that are extremely long (approaching 20k characters) there is an additional filter leaving out contextes having over 3000 characters. Note that there is an initial filter only including

special categories like Summarization, Closed Question Answering and Information Extraction. Since two different extraction models were included in tests, the evaluation happens on two datasets. They differ by the model used for extracting triplets from contexts: REBEL and Custom Trained Extractor on WebNLG that is introduced in this project.

The average Rouge F1 score on the data points calculates each model's final score. Rouge F1 scores are calculated from Rouge recall and Rouge precision (their formulas are discussed in Chapter 2.8).

$$F1 = \frac{2 \times Precision \times Recall}{(Precision + Recall)}$$

Figure 4.5 Formula of the Rouge F1 score. Precision and Recall keywords refer to the Rouge Precision and Rouge Recall formulas discussed in Chapter 2.8

Table.4.2 shows the performance of different models on triplet comprehension tasks using the Dolly dataset in terms of ROUGE scores. The table compares pre-trained to fine-tuned models and shows that fine-tuned models perform better, especially in phrasal accuracy and structural coherence, as indicated by higher ROUGE-2 and ROUGE-L scores. The models are divided into two sections: those using the REBEL extraction model and others with a custom fine-tuned extractor.

	model	extraction model	rouge-1 score	rouge-2 score	rouge-L score
1	Meta/Llama-2 7b hf	rebel	0.35	0.12	0.27
2	Google/gemma 7b	rebel	0.39	0.15	0.32
3	Google/gemma-2b	rebel	0.41	0.15	0.34
4	Google/gemma-2b-it	rebel	0.46	0.19	0.37
5	Google/gemma-7b-it	rebel	0.46	0.19	0.39
6	Meta/Llama-2 7b chat hf	rebel	0.52	0.20	0.40
7	Mistralai/Mistral 7B Instruct v0.2	rebel	0.59	0.28	0.48
8	finetuned Google/gemma 7, b it	rebel	0.57	0.23	0.46
9	finetuned Google/gemma 2b it	rebel	0.57	0.23	0.47
10	finetuned Mistralai/Mistral 7B Instruct v0.2	rebel	0.61	0.25	0.47
11	finetuned-fullCollator gemma-2b-it	trained extractor on WebNLG	0.58	0.29	0.49
12	finetuned-completionsOnlyCollator gemma-2b-it	trained extractor on WebNLG	0.62	0.33	0.52

The evaluation is done with both out-of-the-box pre-trained models and their aligned versions. The variations in their performance on each metric are significant. The rouge-1 score ranges from 0.35 to 0.62, which measures the average ratio of similar words between the reference and machine-generated texts. The rouge-2 score ranges from 0.12 to 0.33, which measures the identical bigrams between reference and generated text and can be considered natural based on the rouge-1 scores. The rouge-L score ranges from 0.27 to 0.52, which compares the longest common subsequence (not necessarily contiguous) to the full length of the reference. Having an average of 52% of the subsequent words identical to the reference is a strong performance, taking into account that a sentence has numerous variations of wordings with identical semantics. Having an average of 62% of identical words and 52% of subsequent words identical to the reference text also indicates that the model utilises the externally injected triplet data and performs text generation based on the words inside the triplets.

For an example, let's take the following reference text: "*The cat sat on the mat*". When we look at the generated text: "*A dog lay on the floor*", the calculated rouge score compared to the reference text would be rouge-1: 0.5 because half of the words are matching; Rouge-2: 0.2 because the only bigram that matches is "on the", finally rouge-L would be 0.5 because the LCS is "The on the" which is half of the full length of the reference text. Other similar examples include "The cat was sitting on the mat" with rouge-1, rouge-2 and rouge-L scores: 0.83, 0.6, 0.77.

The Llama-2-7B-HF-Chat version exhibited differing performance compared to its simpler Llama-2-7B-HF counterpart. The "chat" keyword indicates that the model was further fine-tuned on dialogues and therefore, better handles the flow of conversations, which indicates a better performance in triplet comprehension tasks.

The Mistral-7B-Instruct model performed remarkably well, largely due to its specialisation based on fine-tuning through instructions. This variant of the Mistral model was specifically oriented toward handling detailed instructions with great proficiency in executing tasks accurately and following context. This ability is of particular use in the handling of structured data, like triplets, where the generation of precise responses with regard to relational data is required.

When fine-tuned, the Gemma and Mistralai models matched incredibly well after just 50 training steps (See Figure 4.3 for Gemma loss when fine-tuning), posting large gains in the ROUGE-1 and ROUGE-L scores. These improvements indicate a greater lexical overlap with reference texts and better structural coherence in generated responses. Such rapid alignment underlines the robust architecture of the models and how well they could be adapted to specific datasets and output requirements.

The evaluation task was triplet comprehension, using a dataset prepared by the Rebel model, then replaced by a custom trained relation extractor on WebNLG. REBEL is inherently limited to around 200 relational categories, which might degrade the comprehension task's quality. The model might not capture all the necessary contextual nuances. Thus, the performance of other models on this dataset would be constrained by incomplete or over-simplification of relational data provided by Rebel. However, the

adoption of the trained relation extractor provided triplets that contained more detail and eloquent words compared to REBEL. The slightly better performance of models on the new dataset indicates that this is the case.

Fine-tuned Gemma 2b performed relatively better when evaluated on the Dolly dataset, which was processed using the custom triplet extractor. (See improved rouge scores on Table 4.2; rows 11, 12.) This, in a way, points to the fact that the much richer and more contextually nuanced triplet contexts generated by the custom triplet extractor contributed to models' enhanced capabilities of understanding and responding.

Notice the difference between the models' performances using the *Full Collator* and those using the *Completions Only Collator*. The latter is trained on expected answers only and not on full prompts. Such specialisation suggests that the Completions Only Collator model can focus harder on the evaluation task, potentially leading to more targeted and effective response generation.

4.5 Model Responses on Cherry Picked Task

In this section, I demonstrate a sample run on one of the data points inside the evaluation set. The selected models have to answer a question with structured triplet data that is handled to them. The input prompt is represented in the table below, signaling the position of the user input or question, and the provided triplets to the question. The triplets for this example are extracted with the WebNLG based custom extracter.

Table 4.3 Table representing a sample datapoint of the Dolly dataset generated by the custom relation extracter. The table indicates the user input which usually an instruction or this time a question. The following content is the provided triplets to the model, creating a context to the question.

User input:		
What is the Fallout game?		
Relations (triplets):		
head	**type**	**tail**
Fallout:_A_Post-Nuclear_Role-Playing_Game	prehistoricdeception	Vault Dweller infiltrates Vault 13, looking for supplies
		Nuclear_War
	publisher	Interplay_Productions
	released	1997
	setting	Underground shelter Vault 13, a haven for the Commonwealth of Nations
		Underground, a decade after World War III
United Stated	nuclearwar	China
Vault_13	location	Vault_13
	status	Incapacitated, but not deactivated
		Operated by the Commonwealth of Nations

The selected input is given to two different models: a pre-trained text generation model Google/Gemma-2b-it, and an already aligned model based on MistralAI's Mistral Instruct v0.2 model.

Table 4.4 This table shows the generated answers by each selected models. The input prompt is represented in Table 4.3.

Google/Gemma-2b-it **Not Aligned**	It is a video game played as a **role-playing game**, which is a video game in which players assume the role of a character. The player character must then travel to multiple environments to complete quests (and to avoid being killed by dangerous enemies) while also fighting against a world possessed by tyrannical government entities. With that in mind, there are many unique characteristics that make Fallout unique. The world that Fallout takes place in is a wasteland, and the player character uses an RPG style combat system, and the game features a large number of crafting options (weapons, armour, electronics etc.). \ The game is in the cyberpunk genre with lots of gunplay. The game is developed by Bethesda Game Studios.
MistalAI/Mistral-7B-v02 **Aligned**	The Fallout is a **role-playing video game**. In its background, it depicts a **fictional nuclear war** between the **United States and China**, which devastated the planet causing the "Nuclear Winter," a dark, apocalyptic environment, The game puts the player in the role of "**Vault-Dweller**," a survivor who was rescued by a bunker named **Vault-13** before **World War III**. The game lets the player live as a wanderer exploring ruins and interacting with other survivors. The first game, **Fallout: A Post-Nuclear Role-Playing Game**, was published and released in **1997 by Interplay** and is considered a classic in the role-playing game genre.

The Table 4.4 above shows the different answers from the models to the received input prompt in Table 4.3. The main difference between the baseline pre-trained and the fine-tuned model is using the injected triplet data. In the responses, each part of the text that can be connected to the provided triplets is highlighted. While a pre-trained model barely utilises the externally provided data, the aligned model's text generation can be controlled more easily by specifying a domain knowledge it can base its generation on.

Chapter 5: Conclusions

In conclusion, this thesis has explored the potential of integrating graph databases with language models to extend the limited context window typically found in transformer-based systems. This integration also helps enrich conversational AI knowledge by providing a structured form of knowledge. It is also in line with the techniques used in projects like MemGPT, which uses external databases to improve context awareness without sacrificing speed.

Data injection into prompts has been tested and proved useful. The models are able to use the structured information from the graph databases in the form of triplets directly in the prompts to generate more contextually aware and semantically rich responses. This approach is more natural to human cognition as it enables the models to refer to a wider context outside the conversation.

Moreover, the proper use of triplet data has been extremely helpful for these models to connect with graph databases. This compatibility is crucial for the development of systems that need a large knowledge base as it provides dynamic access to a large amount of information, thus increasing the scope of language models' work. With the help of nodes and edges, we can represent data in a different way and improve querying and data retrieval processes.

Projects like MemGPT have shown how external databases can effectively extend the context window. By adopting a similar approach but using graph databases, this thesis contributes to the ongoing discussion on how to mitigate the limitations of current language models, particularly in terms of their traditionally constrained memory and context capabilities.

References

[1] J. H. M. Daniel Jurafsky, „Speech and Language Processing An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition Third Edition draft,” 2023, p. 32.

[2] R. M. Schmidt, „Recurrent Neural Networks (RNNs): A gentle Introduction and Overview,” 2019. [Online]. Available: https://arxiv.org/abs/1912.05911. [Hozzáférés dátuma: 2024].

[3] E. R. M. Ralf C. Staudemeyer, „Understanding LSTM -- a tutorial into Long Short-Term Memory Recurrent Neural Networks,” [Online]. Available: https://arxiv.org/abs/1909.09586. [Hozzáférés dátuma: 2024].

[4] N. S. N. P. J. U. L. J. A. N. G. L. K. I. P. Ashish Vaswani, „Attention Is All You Need,” [Online]. Available: https://arxiv.org/abs/1706.03762. [Hozzáférés dátuma: 2024].

[5] R. M. C. S. G. S. Y. G. S. P. K. R. M. D. R. G. R. H. J. P. B. W. W. A. V. V. T. R. G. Gokul Yenduri, „Generative Pre-trained Transformer: A Comprehensive Review on Enabling Technologies, Potential Applications, Emerging Challenges, and Future Directions,” 2023. [Online]. Available: https://arxiv.org/abs/2305.10435. [Hozzáférés dátuma: 2024].

[6] M.-W. C. K. L. K. T. Jacob Devlin, „BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” 2018. [Online]. Available: https://arxiv.org/abs/1810.04805. [Hozzáférés dátuma: 2024].

[7] K. C. G. C. J. D. Tomas Mikolov, „Efficient Estimation of Word Representations in Vector Space,” 2013. [Online]. Available: https://arxiv.org/abs/1301.3781. [Hozzáférés dátuma: 2024].

[8] R. S. C. M. Jeffrey Pennington, „GloVe: Global Vectors for Word Representation,” 2014. [Online]. Available: https://aclanthology.org/D14-1162/. [Hozzáférés dátuma: 2024].

[9] E. G. A. J. T. M. Piotr Bojanowski, „Enriching Word Vectors with Subword Information,” 2017. [Online]. Available: https://arxiv.org/abs/1607.04606. [Hozzáférés dátuma: 2024].

[10] A. L. M. J. Jean-Baptiste Cordonnier, „Multi-Head Attention: Collaborate Instead of Concatenate,” 2020. [Online]. Available: https://arxiv.org/abs/2006.16362. [Hozzáférés dátuma: 2024].

[11] H. T. S. B. H. W. C. Y.-W. C. C.-S. F. Pu-Chin Chen, „A Simple and Effective Positional Encoding for Transformers,” 2021. [Online]. Available: https://arxiv.org/abs/2104.08698. [Hozzáférés dátuma: 2024].

[12] J. W. R. C. D. L. D. A. I. S. Alec Radford, „Language Models are Unsupervised Multitask Learners,” 2019. [Online]. Available: https://paperswithcode.com/paper/language-models-are-unsupervised-multitask. [Hozzáférés dátuma: 2024].

[13] B. M. N. R. M. S. J. K. P. D. A. N. P. S. G. S. A. A. S. A. A. H.-V. G. K. T. H. R. C. A. R. D. Tom B. Brown, „Language Models are Few-Shot Learners,” 2020. [Online]. Available: https://arxiv.org/abs/2005.14165. [Hozzáférés dátuma: 2024].

[14] S. Sukhbaatar, E. Grave, P. Bojanowski és A. Joulin, „Adaptive Attention Span in Transformers,” 2019. [Online]. Available: https://arxiv.org/abs/1905.07799. [Hozzáférés dátuma: 2024].

[15] C. M. Y. L. Liang Yao, „KG-BERT: BERT for Knowledge Graph Completion,” 2019. [Online]. Available: https://paperswithcode.com/paper/kg-bert-bert-for-knowledge-graph-completion. [Hozzáférés dátuma: 2024].

[16] Z. L. M. S. Y. L. X. Z. Yankai Lin, „Learning Entity and Relation Embeddings for Knowledge Graph Completion,” 2015. [Online]. Available: https://paperswithcode.com/paper/learning-entity-and-relation-embeddings-for. [Hozzáférés dátuma: 2024].

[17] N. T. P. H. T. C. Urchade Zaratiana, „An Autoregressive Text-to-Graph Framework for Joint Entity and Relation Extraction,” 2024. [Online]. Available: https://arxiv.org/abs/2401.01326. [Hozzáférés dátuma: 2024].

[18] „Hugging Face Documentation,” [Online]. Available: https://huggingface.co/docs. [Hozzáférés dátuma: 2024].

[19] S. W. K. L. V. F. S. G. P. I. S. J. E. G. Charles Packer, „MemGPT: Towards LLMs as Operating Systems,” 2023. [Online].

[20] H. D. A. R. E. W. C. R. Nikhil Kandpal, „Large Language Models Struggle to Learn Long-Tail Knowledge,” 2023. [Online]. Available: https://arxiv.org/abs/2211.08411. [Hozzáférés dátuma: 2024].

[21] G. T. a. T. M. a. C. H. a. R. D. a. S. B. a. S. P. a. L. S. a. M. R. a. M. S. K. a. J. L. a. P. T. a. L. H. a. P. G. S. a. Aaka, „Gemma: Open Models Based on Gemini Research and Technology," 2024. [Online]. Available: https://arxiv.org/abs/2403.08295. [Hozzáférés dátuma: 2024].

[22] H. T. a. L. M. a. K. S. a. P. A. a. A. A. a. Y. B. a. N. B. a. S. B. a. P. B. a. S. B. a. D. B. a. L. B. a. C. C. F. a. M. Chen, „Llama 2: Open Foundation and Fine-Tuned Chat Models," 2023. [Online]. Available: https://arxiv.org/abs/2307.09288. [Hozzáférés dátuma: 2024].

[23] A. Q. J. a. A. S. a. A. M. a. C. B. a. D. S. C. a. D. d. l. C. a. F. B. a. G. L. a. G. L. a. L. S. a. L. R. L. a. M.-A. Lachau, „Mistral 7B," 2023. [Online]. Available: https://arxiv.org/abs/2310.06825. [Hozzáférés dátuma: 2024].

[24] P. S. a. T. N. a. I. Bhattacharya, „Do the Benefits of Joint Models for Relation Extraction Extend to Document-level Tasks?," 2023. [Online]. Available: https://aclanthology.org/2023.ijcnlp-short.6.pdf. [Hozzáférés dátuma: 2024].

[25] Z. a. J. Z. a. T. K. Yan, „An Empirical Study of Pipeline vs. Joint approaches to Entity and Relation Extraction," 2022. [Online]. Available: https://aclanthology.org/2022.aacl-short.55. [Hozzáférés dátuma: 2024].

[26] Z. C. a. C. Guo, „A pattern-first pipeline approach for entity and relation extraction," 2022. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0925231222004489. [Hozzáférés dátuma: 2024].

[27] T. Bratanic, „Extract knowledge from text: End-to-end information extraction pipeline with spaCy and Neo4j," 2022. [Online]. Available: https://towardsdatascience.com/extract-knowledge-from-text-end-to-end-information-extraction-pipeline-with-spacy-and-neo4j-502b2b1e0754. [Hozzáférés dátuma: 2024].

[28] R. N. Pere-Lluís Huguet Cabot, „REBEL: Relation Extraction By End-to-end Language generation," 2021. [Online]. Available: https://aclanthology.org/2021.findings-emnlp.204/. [Hozzáférés dátuma: 2024].

[29] databricks, „databricks-dolly-15k," 2022. [Online]. Available: https://huggingface.co/datasets/databricks/databricks-dolly-15k. [Hozzáférés dátuma: 2024].

[30] A. S. S. N. L. P.-B. Claire Gardent, „Creating Training Corpora for NLG Micro-Planners," 2017. [Online]. Available: https://doi.org/10.18653/v1/P17-1017. [Hozzáférés dátuma: 2024].

[31] G. e. al., „WebNLG," 2017, [Online]. Available: https://paperswithcode.com/dataset/webnlg. [Hozzáférés dátuma: 2024].

[32] C.-Y. Lin, „ROUGE: A Package for Automatic Evaluation of Summaries,” 2004. [Online]. Available: https://aclanthology.org/W04-1013/. [Hozzáférés dátuma: 2024].

[33] N. Schluter, „The limits of automatic summarisation according to ROUGE,” 2017. [Online]. Available: https://aclanthology.org/E17-2007/. [Hozzáférés dátuma: 2024].

[34] HuggingFace, „What is the ROUGE metric?,” 2022. [Online]. Available: https://www.youtube.com/watch?v=TMshhnrEXlg. [Hozzáférés dátuma: 2024].

[35] J. W. a. M. B. a. V. Y. Z. a. K. G. a. A. W. Y. a. B. L. a. N. D. a. A. M. D. a. Q. V. Le, „Finetuned Language Models Are Zero-Shot Learners,” 2022. [Online]. Available: https://arxiv.org/abs/2109.01652. [Hozzáférés dátuma: 2024].

[36] S. Z. a. L. D. a. X. L. a. S. Z. a. X. S. a. S. W. a. J. L. a. R. H. a. T. Z. a. F. W. a. G. Wang, „Instruction Tuning for Large Language Models: A Survey,” 2024. [Online]. Available: https://arxiv.org/abs/2308.10792.

[37] H. Face, „Templates for Chat Models,” [Online]. Available: https://huggingface.co/docs/transformers/main/chat_templating#templates-for-chat-models. [Hozzáférés dátuma: 2024].

[38] H. F. Meta, „Llama-2 Intended Use,” [Online]. Available: https://huggingface.co/meta-llama/Llama-2-7b-chat-hf#intended-use. [Hozzáférés dátuma: 2024].

[39] H. F. Google, „Gemma Chat Template,” [Online]. Available: https://huggingface.co/google/gemma-2b-it#chat-template. [Hozzáférés dátuma: 2024].

[40] huggingface, „TRL - Transformer Reinforcement Learning,” [Online]. Available: https://github.com/huggingface/trl, https://huggingface.co/docs/trl/index.

[41] „Weights and Biases,” [Online]. Available: https://wandb.ai/. [Hozzáférés dátuma: 2024].

[42] Wikipedia, „Napoleon,” [Online]. Available: https://en.wikipedia.org/wiki/Napoleon. [Hozzáférés dátuma: 2024].

[43] T. Dao, D. Y. Fu, S. Ermon, A. Rudra és C. Ré, „FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022. [Online]. Available: https://arxiv.org/abs/2205.14135. [Hozzáférés dátuma: 2024].

[44] S. K. B. C. M. Z. M. T. A. H. H. A. D. K. Benoit Jacob, „Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," 2017. [Online]. Available: https://arxiv.org/abs/1712.05877. [Hozzáférés dátuma: 2024].

[45] M. Z. B. C. D. K. W. W. T. W. M. A. H. A. Andrew G. Howard, „MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017. [Online]. Available: https://arxiv.org/abs/1704.048612. [Hozzáférés dátuma: 2024].

[46] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li és L. Fei-Fei, „ImageNet: A large-scale hierarchical image database," 2009. [Online]. Available: https://ieeexplore.ieee.org/document/5206848. [Hozzáférés dátuma: 2024].

[47] Y. S. P. W. Z. A.-Z. Y. L. S. W. L. W. W. C. Edward J. Hu, „LoRA: Low-Rank Adaptation of Large Language Models," 2021. [Online]. Available: https://arxiv.org/abs/2106.09685. [Hozzáférés dátuma: 2024].

[48] P. SEBASTIAN RASCHKA, „Improving LoRA: Implementing Weight-Decomposed Low-Rank Adaptation (DoRA) from Scratch," 2024. [Online]. Available: https://magazine.sebastianraschka.com/p/lora-and-dora-from-scratch. [Hozzáférés dátuma: 2024].

[49] D. Schuurmans, „Memory Augmented Large Language Models are Computationally Universal," 2023. [Online]. Available: https://arxiv.org/abs/2301.04589. [Hozzáférés dátuma: 2024].

[50] Y. J. K. J. B. HyunJin Kim, „PEMA: Plug-in External Memory Adaptation for Language Models," 2023. [Online]. Available: https://www.microsoft.com/en-us/research/publication/pema-plug-in-external-memory-adaptation-for-language-models/. [Hozzáférés dátuma: 2024].

[51] Y. L. N. G. M. G. A. M. O. L. V. S. L. Z. Mike Lewis, „BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," 2019. [Online]. Available: https://arxiv.org/abs/1910.13461.

[52] J. W. X. J. D. A. C. L. W. P. M. C. Z. S. A. K. S. A. R. J. S. J. H. F. K. L. M. M. S. A. A. P. W. P. C. Long Ouyang, „Training language models to follow instructions with human feedback," 22. [Online]. Available: https://arxiv.org/abs/2203.02155. [Hozzáférés dátuma: 2024].

[53] M. C. a. M. H. a. A. M. a. J. X. a. J. W. a. S. S. a. A. G. a. P. W. a. M. Z. a. R. Xin, „Free Dolly: Introducing the World's First Truly Open Instruction-Tuned LLM," 2023. [Online]. Available: https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm. [Hozzáférés dátuma: 2024].

List of Figures

Figure 2.1 Illustration of a Natural Language Processing (NLP) pipeline. [27] 10

Figure 2.2 An example inference of a relation extraction pipeline. [27] 10

Figure 2.3 REBEL's algorithm to transform a set of triplets into text format. This transformation is necessary to create a valid output set for fine-tuning the BART model. Notice the special tokens <triplet>, <subj> and <obj> that form a unique prompt template. [28] 11

Figure 2.4 The sentence in the top-left corner is the model's original x input. Below that is the set of triplets corresponding to the input. On the right is the transformed text version of the relation set used to train the model. [28] 11

Figure 2.5 Datapoints from databricks-dolly-15k 13

Figure 2.6 An instance from the WebNLG dataset containing the written sentence and the corresponding triplet representation of its factual information. (figure from the website [31]) 14

Figure 2.7 Formulas for calculating different variations of ROUGE. Note that LCS stands for longest common sequence. (Image taken from this video: [34]) 15

Figure 3.1 The diagram illustrates the architecture of the multi-agent system mentioned, which is designed to enhance conversational AI by integrating with an external graph database. It shows three specialised agents: the first agent extracts useful triplet data from user input, the second identifies keywords, and the third, a conversational AI agent, uses the extracted information to generate informed responses. 17

Figure 3.2 shows the Architecture of the second implementation. Notice that keyword extraction from text happens by querying each part of the triplets. 21

Figure 3.3 Sample datapoint of the preprocessed WebNLG dataset with combined sentences. Notice the grammatical order of the triplets. 22

Figure 3.4 This is a sample prompt from the fine-tuning of the relation extractor model. Notice the special tokens: ### Text:, ### Relations:, <entry>, <head>, <tail>, <rel>. During fine-tuning the model, weights are calculated for the generated token sequences only after the ### Relations: token. 23

Figure 3.5 A sample data point from Dolly's context field was transferred into a set of triplets, forming a new dataset. 24

Figure 4.1 pre-trained **google/gemma-2b-it** model's response to a instruction prompt with triplets inserted. 26

Figure 4.2 **Fine-tuned Google/gemma-2b-it** model's response to an instruction prompt. Notice that the response contains a word from the provided triplets (Amazon biome) in this aligned version. This indicates that the aligned model utilises the externally injected data. 27

Figure 4.3 Error scores during fine-tuning on Dolly. The graph contains a comparison between two training runs using a complete collator (red) and a completions-only collator (green). Notice that the models aligned after 30 learning steps. (Plot created in WandB [41]).....28

Figure 4.4 shows the training loss for the Google/Gemma 2b-it model over 5000 steps in a logarithmic scale to demonstrate how the model performance improves while training on a custom dataset created based on the WebNLG data. The loss values provided by the graph imply that the model is constantly learning and improving as the training continues. (Plot created in WandB [41]) 29

Figure 4.5 Formula of the Rouge F1 score. Precision and Recall keywords refer to the Rouge Precision and Rouge Recall formulas discussed in Chapter 2.8 31

Figure 0.1: (From [42]) **Left**: FlashAttention uses tiling to prevent materialisation of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FlashAttention loops through blocks of the K and V matrices and loads them to fast on-chip SRAM. FlashAttention loops over blocks of the Q matrix (blue arrows) in each block, loading them to SRAM and writing the output of the attention computation back to HBM. **Right**: Speedup over the PyTorch implementation of attention on GPT-2. FlashAttention does not read and write the large $N \times N$ attention matrix to HBM, resulting in a 7.6× speedup in the attention computation. 48

Figure 0.2 (From [43]**): Integer-arithmetic-only quantisation**. a) Integer-arithmetic-only inference of a convolution layer. The input and output are represented as 8-bit integers according to equation 1. The convolution involves 8-bit integer operands and a 32-bit integer accumulator. The bias addition involves only 32-bit integers. The ReLU6 nonlinearity only involves 8-bit integer arithmetic. b) Training with simulated quantisation of the convolution layer. All variables and computations are carried out using 32-bit floating-point arithmetic. Weight quantisation ("wt quant") and activation quantisation ("act quant") nodes are injected into the computation graph to simulate the effects of the quantisation of the variables. The resultant graph approximates the integer-arithmetic-only computation graph in panel a) while being trainable using conventional optimisation algorithms for floating point models. c) Our quantisation scheme benefits from the fast integer-arithmetic circuits in common CPUs to deliver an improved latency-vs-accuracy tradeoff. The figure compares integer quantised MobileNets [44] against floating point baselines on ImageNet [45] using Qualcomm Snapdragon 835 LITTLE cores. 50

List of Tables

Table 3.1 Propmt templates for the agents. There are instructions describing their task and how to process the user input. Notice that I used one-shot prompts in these examples. Also, each prompt has a frame with special tokens; this is further discussed in Chat Templates. 18

Table 3.2 Recommended templates for using the specified models as conversational agents. Notice that each template incorporates specific formatting and special tokens to structure interactions appropriately. The <bos> and <eos> tokens represent the beginning and end of sequences, helping the models to delineate the start and end of user and model turns effectively. Special tokens such as [INST] and <<SYS>> are used to indicate instructional or system-specific prompts [38] [39]. 19

Table 4.1 Comparison of the relation extracters: REBEL and the custom trained extractor on WebNLG. The table shows their extracted relations from the first paragraph of Napoleon I's Wikipedia page [42]. 30

Table.4.2 shows the performance of different models on triplet comprehension tasks using the Dolly dataset in terms of ROUGE scores. The table compares pre-trained to fine-tuned models and shows that fine-tuned models perform better, especially in phrasal accuracy and structural coherence, as indicated by higher ROUGE-2 and ROUGE-L scores. The models are divided into two sections: those using the REBEL extraction model and others with a custom fine-tuned extractor. 32

Table 4.3 Table representing a sample datapoint of the Dolly dataset generated by the custom relation extracter. The table indicates the user input which usually an instruction or this time a question. The following content is the provided triplets to the model, creating a context to the question. 35

Table 4.4 This table shows the generated answers by each selected models. The input prompt is represented in Table 4.3. 36

Appendix A: Environment Configuration

The following tools and technologies were utilised to process my experiments faster, monitor them, fine-tune my models with memory efficiency and make the workflow more fluid overall.

A.1 Flash Attention [42]

Although the transformers play major roles in natural language processing and image classification, making them the best candidates for complex operation tasks, they face a major challenge of slowdowns and high memory usage when processing long sequences because it has quadratic complexity. The intervention called FlashAttention evolved as the new approach to this problem by implementing a modified exact attention mechanism, which considers the information-output component's process. It is essentially the same as the more advanced versions that cut down the number of memory reads/writes from the HBM and move it to the local SRAM, leading to increased overall memory efficiency.

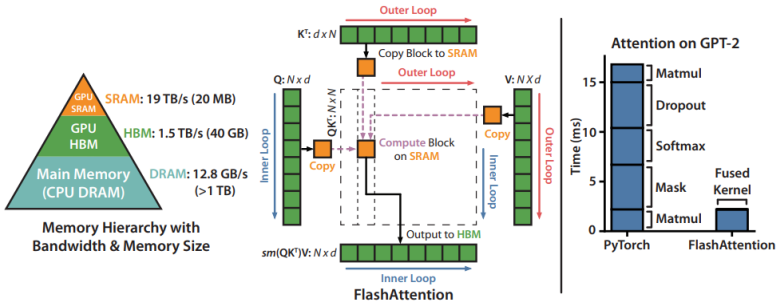


*Figure 0.1: (From [42]) **Left**: FlashAttention uses tiling to prevent materialisation of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FlashAttention loops through blocks of the K and V matrices and loads them to fast on-chip SRAM. FlashAttention loops over blocks of the Q matrix (blue arrows) in each block, loading them to SRAM and writing the output of the attention computation back to HBM. **Right**: Speedup over the PyTorch implementation of attention on GPT-2. FlashAttention does not read and write the large $N \times N$ attention matrix to HBM, resulting in a 7.6× speedup in the attention computation.*

FlashAttention's efficiency partly results from its advanced tiling strategy, which computes attention directly without the actual storage of attention matrices from slow memory. This technique reduces the number and size of transfers between the memory levels. It saves the full strength of the attention mechanisms because of the precise calculations, unlike those proxies that lessen the precision for the sake of speed. Through the process of subdivision of the attention matrix into smaller, operable pieces, FlashAttention successfully decreases the I/O complexity and maximises manufacturing speed.

Unlike another attention block, FlashAttention brings in longer sequences than is usual for Transformer models and does so with much lower computational consumption. This extension rapidly brings attention to computations on only relevant parts of data, which consequently plays a vital role in speeding up training and inference times. This effect is achieved with negligible effects on model accuracy and efficiency.

According to the experiment, FlashAttention is way ahead of the traditional attention mechanism and allows speed models like BERT for up to 15% and in GPT-2 setups, even about a three-time speed. It involves processing longer dependencies in a resource-efficient manner to reduce perplexity and increase classification accuracy for long documents. Today, thanks to this progress, Transformers have unleashed their superpowers to solve tough problems like the Path-X challenge, thus moving the boundaries and defeating tasks that were previously out of their reach due to memory constraints.

Using these inventions, a deep recognition system called FlashAttention raises the fundamental limits of Transformer models in handling long sequences. It presents new records in the number of operations and storage consumption.

A.2 Quantization [43]

Due to the widespread usage of intelligent mobile devices and the huge computational requirements of deep learning models, intelligent solutions for on-device inference are necessary. This paper [43] proposes an integer single-precision 2-multiplier without a shifter, which is optimised for integer-only arithmetic and can improve the performance of devices that provide integer operations only. The technique works side-by-side to establish a training protocol that allows the model to stay sharp after complete quantisation. A drop in both the error rate and the total memory that is required for processing was clearly observed, which was particularly desirable for mobile networks because MobileNets are specifically optimised for these devices. The results demonstrated improvement in benchmarking, which is evident in standard tasks such as ImageNet detection and COCO tasks. Further, they showed the practical advantages of the mentioned quantisation approach in ubiquitous mobile CPUs.

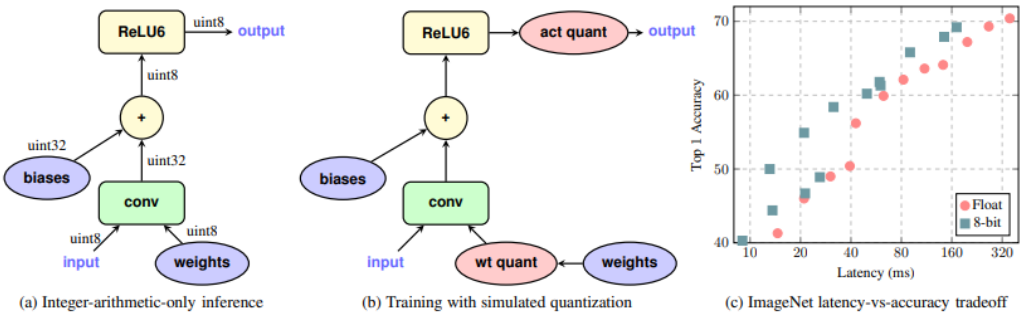


*Figure 0.2 (From [43])**: Integer-arithmetic-only quantisation**. a) Integer-arithmetic-only inference of a convolution layer. The input and output are represented as 8-bit integers according to equation 1. The convolution involves 8-bit integer operands and a 32-bit integer accumulator. The bias addition involves only 32-bit integers. The ReLU6 nonlinearity only involves 8-bit integer arithmetic. b) Training with simulated quantisation of the convolution layer. All variables and computations are carried out using 32-bit floating-point arithmetic. Weight quantisation ("wt quant") and activation quantisation ("act quant") nodes are injected into the computation graph to simulate the effects of the quantisation of the variables. The resultant graph approximates the integer-arithmetic-only computation graph in panel a) while being trainable using conventional optimisation algorithms for floating point models. c) Our quantisation scheme benefits from the fast integer-arithmetic circuits in common CPUs to deliver an improved latency-vs-accuracy tradeoff. The figure compares integer quantised MobileNets [44] against floating point baselines on ImageNet [45] using Qualcomm Snapdragon 835 LITTLE cores.*

The standard state-of-the-art CNNs are proven to be very accurate but, at the same time, are quite large and might be impossible to run on mobile phones designed for larger devices due to the larger number of data they process. The article emphasises the trend toward developing models that balance the need for efficiency and effective memory usage against the need for speed in mobile platforms where both limited memory and quick processing are critical. It discusses two primary strategies in model optimisation: applying architectures that are configured for fewer resources instead of inventing brand-new ones. Also, by quantising already developed models, their numerical precision would be reduced, thereby making both the model size and running time demands a lot more than they currently are.

The main objective of the proposed quantisation framework is to transform mobiles from floating-point to integer-only arithmetic. The main reason is the fact that most popular mobile hardware lacks floating-point support. This quantisation process provides for

adjustments that are exact on the level of the network's weights and activations, transforming them into 8-bit integers, with 32-bit integers for bias vectors as the scope. These adaptations create a system that operates using simpler, faster integer arithmetic and also suits environments where computations executed in single-precision are performed, transforming such an environment into one that uses higher resource architecture.

To mitigate the loss of model performance after quantisation, the authors designed a training protocol that mimics the quantisation effects in the training phase. Thus, this technique can be used to pre-optimize the model by anticipating the effects of the quantisation constraint. The training adjustments are made by quantising weights and activations in the training process and fine-tuning the model to achieve minimum accuracy loss. The deployment is a great asset that can exploit the available hardware resources, confirming that the integer-only quantisation can be used to achieve near-original model performance with reduced computational requirements.

The practical aspects of the quantisation method are confirmed by the fact that it yields significant improvements in the latency-accuracy trade-offs on all-purpose mobile CPUs. Benchmarks on ImageNet and COCO with MobileNets demonstrate significant improvement, which confirms that integer-only quantisation decreases computational cost and saves energy. In some cases, it increases the accuracy of complex models on resource-restricted devices. This breakthrough indicates a way to scale up the deployment of neural networks directly into a broad range of mobile devices, which in turn may dramatically change the way deep learning models are used in the mobile setting.

A.3 PEFT and LoRA

Low-rank adaptation (LoRA) [46] [47] is a machine learning method that enhances a pre-trained model, such as an LLM or vision transformer, to better fit a particular dataset, usually a smaller one. This is achieved by adjusting only a low-rank subset of the model's parameters. This is an important strategy for cost reduction in fine-tuning because it facilitates the effective adjustment of large models.

LLMs, being of considerable size, can incur high costs in terms of GPU memory limits while updating all model weights during training. Assume that we possess a substantial weight matrix W for a specific layer. During the backpropagation process, we acquire knowledge about a **ΔW** matrix, which has data on the extent to which we should modify the initial weights to reduce the loss function while training. The weight update is defined as follows in regular training and fine-tuning:

$$\mathbf{W}_{\text{updated}} = \mathbf{W} + \mathbf{\Delta W}$$

The LoRA method, as suggested in reference [46], provides a more efficient approach for computing weight updates ΔW by learning an approximation of it, $\mathbf{\Delta W} \approx \mathbf{A.B}$. To clarify, in LoRA, we have the following components: A and B, which are both tiny weight matrices. (A.B stands for matrix multiplication of A and B.)

$$\mathbf{W}_{\text{updated}} = \mathbf{W} + \mathbf{A.B}$$

If a trained weight matrix **W** has dimensions of 1,000×1,000, then the weight update matrix **ΔW** in regular finetuning also has dimensions of 1,000×1,000. In this scenario, **ΔW** has 1,000,000 parameters. If we consider a LoRA rank of 2, then A is a 1000×2 matrix, and B is a 2×1000 matrix, and we only have 2×2×1,000 = 4,000 parameters that we need to update when utilising LoRA. In the prior example, with a rank of 2, that's 250 times less parameters.